GS Dheekshith
PES2UG24CS166

OS-VCS Orange Problem Statement:

Phase 1: Object Storage Foundation

This phase focuses on building a content-addressable storage system using hashing. Objects are stored with integrity checks and organized efficiently using directory sharding.

```
dheekshith@asus-a15:~/PES2UG24CS166-PES-VCS$ make test_objects
./test_objects
make: 'test_objects' is up to date.
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
dheekshith@asus-a15:~/PES2UG24CS166-PES-VCS$
```

```
All Phase 1 tests passed.
dheekshith@asus-a15:~/PES2UG24CS166-PES-VCS$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
dheekshith@asus-a15:~/PES2UG24CS166-PES-VCS$
```

Phase 2: Tree Objects

This phase implements directory structures using tree objects. It converts the index into a hierarchical representation that captures the project snapshot.

```
dheekshith@asus-a15:~/PES2UG24CS166-PES-VCS$ make test_tree
./test_tree
make: 'test_tree' is up to date.
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
dheekshith@asus-a15:~/PES2UG24CS166-PES-VCS$
```

```
dheekshith@asus-a15:~/PES2UG24CS166-PES-VCS$ find .pes/objects -type f | head -1
dheekshith@asus-a15:~/PES2UG24CS166-PES-VCS$ ./test-objects
bash: ./test-objects: No such file or directory
dheekshith@asus-a15:~/PES2UG24CS166-PES-VCS$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
dheekshith@asus-a15:~/PES2UG24CS166-PES-VCS$ xxd $(find .pes/objects -type f | head -1) | head -20
00000000: 626c 6f62 2031 3500 5465 7374 2069 6e74  blob 15.Test int
00000010: 6567 7269 5879 0a                        egriXy.
```

Phase 3: The Index (Staging Area)

This phase introduces the staging area that tracks changes before committing. It manages file metadata, detects changes, and maintains a consistent index file.

```
dheekshith@asus-a15:~/PES2UG24CS166-PES-VCS$ make pes
./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
make: 'pes' is up to date.
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  tree.c
  untracked:  tree.h
  untracked:  index.c
  untracked:  test_tree
  untracked:  test_tree.c
  untracked:  pes.h
  untracked:  Makefile
  untracked:  commit.c
  untracked:  pes.c
  untracked:  commit.h
  untracked:  object.c
  untracked:  test_sequence.sh
  untracked:  .gitignore
  untracked:  index.h
  untracked:  README.md
  untracked:  test_objects.c
  untracked:  test_objects

dheekshith@asus-a15:~/PES2UG24CS166-PES-VCS$
```

```
  untracked:  index.c
  untracked:  test_tree
  untracked:  test_tree.c
  untracked:  pes.h
  untracked:  Makefile
  untracked:  commit.c
  untracked:  pes.c
  untracked:  commit.h
  untracked:  object.c
  untracked:  test_sequence.sh
  untracked:  .gitignore
  untracked:  index.h
  untracked:  README.md
  untracked:  test_objects.c
  untracked:  test_objects

dheekshith@asus-a15:~/PES2UG24CS166-PES-VCS$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776707905 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776707905 6 file2.txt
dheekshith@asus-a15:~/PES2UG24CS166-PES-VCS$
```

Phase 4: Commits and History

This phase creates commits that store snapshots of the project along with metadata. It links commits together to form a history and updates references like HEAD.

```
dheekshith@asus-a15:~/PES2UG24CS166-PES-VCS$ ./pes init

echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial Commit"

echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add World"

echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add byebye"

./pes log
Initialized empty PES repository in .pes/
Committed: 88f1853a91b5... Initial Commit
Committed: c21c1e26d0e0... Add World
Committed: 298ede0025bf... Add byebye
commit 298ede0025bf5a35576dbbc32df419ab7e7dbc7591b16901777021988d33f1e2
Author: gsdheekshith <PES2UG24CS166>
Date:   1776708525

    Add byebye

commit c21c1e26d0e076384013d4849493093db1cdf1cca813efab1620e117d2bf1d3c
Author: gsdheekshith <PES2UG24CS166>
Date:   1776708525

    Add World

Add byebye

commit 88f1853a91b5cbb8703fd91e6315679b7cf0fc8d006cf5cdc354c8695b463b23
Author: gsdheekshith <PES2UG24CS166>
Date:   1776708525

    Initial CommittiQ

dheekshith@asus-a15:~/PES2UG24CS166-PES-VCS$
```

```
dheekshith@asus-a15:~/PES2UG24CS166-PES-VCS$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/29/8ede0025bf5a35576dbbc32df419ab7e7dbc7591b16901777021988d33f1e2
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/88/f1853a91b5cbb8703fd91e6315679b7cf0fc8d006cf5cdc354c8695b463b23
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/c2/1c1e26d0e076384013d4849493093db1cdf1cca813efab1620e117d2bf1d3c
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
dheekshith@asus-a15:~/PES2UG24CS166-PES-VCS$
```

```
dheekshith@asus-a15:~/PES2UG24CS166-PES-VCS$ cat .pes/refs/heads/main
cat .pes/HEAD
298ede0025bf5a35576dbbc32df419ab7e7dbc7591b16901777021988d33f1e2
ref: refs/heads/main
dheekshith@asus-a15:~/PES2UG24CS166-PES-VCS$
```

Phase 5: Branching and Checkout (Analysis)

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

To implement pes checkout <branch>, the system would first read the commit hash stored in .pes/refs/heads/<branch> and then update .pes/HEAD so that it points to this branch. After that, it needs to load the commit associated with that hash, extract its tree, and update the working directory so that all files match that snapshot. This means creating new files that exist in the target commit, modifying files that have changed, and removing files that are no longer present. At the same time, the index also has to be updated to reflect this new state. The complexity comes from the fact that three components are being changed together: the HEAD, the index, and the working directory. If this process is not handled carefully, it can overwrite user changes or leave the repository in an inconsistent state.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

To detect a dirty working directory, the system compares the current files with what is recorded in the index. For each tracked file, it first checks basic metadata like size or modification time. If there is a difference, it recomputes the file’s hash and compares it with the hash stored in .pes/index. If the hashes do not match, it means the file has been modified but not committed. Then, this modified file is compared with the version from the target branch using the object store. If switching branches would overwrite these local changes, the checkout operation must be stopped. This way, the system ensures that uncommitted work is not lost.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

In a detached HEAD state, the HEAD points directly to a commit instead of a branch. If a user makes new commits in this state, those commits are created normally, but no branch is updated to point to them. As a result, these commits can become unreachable if the user switches to another branch. However, they are not immediately lost. The user can recover them by creating a new branch that points to the commit, or by using commit history or logs to find the commit hash and reconnect it. Detached HEAD is useful for temporary experimentation, but it requires caution if the changes need to be preserved.

Phase 6: Garbage Collection (Analysis)

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Garbage collection works by identifying which objects are still reachable from the current branches. The process starts from all branch heads in .pes/refs/heads/ and then recursively follows each commit. From each commit, it moves to its parent commits and associated tree objects, and from the trees it follows all referenced blobs. Every object visited during this traversal is marked as reachable. A hash set is ideal for tracking these visited objects because it allows fast lookup and prevents revisiting the same object multiple times. After the traversal is complete, any object that was not marked as reachable can be safely deleted. In a repository with 100,000 commits and 50 branches, the traversal would likely touch all reachable commits along with their trees and blobs, which could easily amount to several hundred thousand objects.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Running garbage collection at the same time as a commit is risky because of possible race conditions. For example, a commit might create a new tree or blob object, but before the commit is fully completed and referenced by HEAD, the garbage collector might run and see that object as unreachable. It could then delete it, even though the commit is about to use it. This would leave the repository in a corrupted state with missing objects. Real Git avoids this by using locking mechanisms to prevent such conflicts, ensuring that objects are written before references are updated, and by introducing a safety window where recently created objects are not immediately eligible for deletion. This coordination prevents accidental loss of valid data.

Repository & Work

GITHUB-repo Link: github_repo-OS-VCS